Project Title: Clinic Appointment System

Phase 3: Database Design and DAO Layer Implementation

◈ 1. Introduction

The Clinic Appointment System is designed to manage interactions between doctors and patients efficiently. This phase focuses on designing a robust relational database schema and implementing a DAO (Data Access Object) layer using JDBC.

The aim is to ensure proper data storage, retrieval, and transaction management while maintaining clean separation between business logic and database operations.

◈ 2. Objectives

- To design a normalized relational database schema
- To establish relationships between entities using primary and foreign keys
- To implement DAO classes for database interaction
- To perform CRUD (Create, Read, Update, Delete) operations
- To implement transaction management using JDBC

◈ 3. Database Design

3.1 Tables Implemented

The following tables are designed:

1. **Doctors**
2. **Patients**
3. **Schedules**
4. **Appointments**
5. **Visit_Records**

3.2 Table Structure

Doctors Table

Column Name	Data Type	Description
doctor_id (PK)	INT	Unique doctor ID
name	VARCHAR	Doctor name
specialization	VARCHAR	Field of expertise
phone	VARCHAR	Contact number
email	VARCHAR	Email address

Patients Table

Column Name	Data Type	Description
patient_id (PK)	INT	Unique patient ID
name	VARCHAR	Patient name
age	INT	Age
gender	VARCHAR	Gender
phone	VARCHAR	Contact number
email	VARCHAR	Email address

Schedules Table

Column Name	Data Type	Description
schedule_id (PK)	INT	Schedule ID
doctor_id (FK)	INT	Reference to doctor
available_date	DATE	Available date
start_time	TIME	Start time
end_time	TIME	End time

Appointments Table

Column Name	Data Type	Description
appointment_id (PK)	INT	Appointment ID

Column Name	Data Type	Description
patient_id (FK)	INT	Reference to patient
doctor_id (FK)	INT	Reference to doctor
schedule_id (FK)	INT	Reference to schedule
appointment_date	DATE	Date of appointment
status	VARCHAR	Status of appointment

Visit Records Table

Column Name	Data Type	Description
record_id (PK)	INT	Record ID
appointment_id (FK)	INT	Reference to appointment
diagnosis	TEXT	Diagnosis details
prescription	TEXT	Medicines prescribed
visit_date	DATE	Date of visit

◈ 4. Entity Relationships

- One doctor can have multiple schedules
- One doctor can handle multiple appointments
- One patient can book multiple appointments
- Each appointment has one visit record

This ensures proper normalization and avoids redundancy.

🔷 5. DAO Layer Implementation

The DAO layer is responsible for interacting with the database. Each entity has a corresponding DAO class:

- DoctorDAO
- PatientDAO
- AppointmentDAO
- ScheduleDAO
- VisitRecordDAO

5.1 Features of DAO Layer

- Encapsulation of SQL queries
- Separation of database logic from UI
- Reusability and maintainability
- Use of JDBC API

◈ 6. CRUD Operations

Each DAO class implements:

- **Create → Insert new records**
- **Read → Fetch records from database**
- **Update → Modify existing data**
- **Delete → Remove records**

Example:

- **Add new patient**
- **Retrieve doctor details**
- **Update appointment status**
- **Delete cancelled appointment**

◈ 7. JDBC Implementation

7.1 Key Concepts Used

- **DriverManager for database connection**
- **PreparedStatement for executing queries**
- **ResultSet for retrieving data**

7.2 Connection Handling

A centralized class is used to manage database connections, improving efficiency and maintainability.

◈ 8. Transaction Management

Transactions are used to ensure data consistency.

Example: Booking an Appointment

Steps:

1. **Check availability**
2. **Insert appointment**
3. **Update schedule**

If any step fails, the entire transaction is rolled back.

◈ 9. Exception Handling

- **SQL exceptions are handled using try-catch blocks**
- **Rollback is performed in case of failure**
- **Input validation is done before database operations**

◈ 10. Advantages of the Design

- **Ensures data integrity using foreign keys**
- **Reduces redundancy through normalization**
- **Improves maintainability with DAO pattern**
- **Provides secure database operations using prepared statements**

◈ 11. Limitations

- **Requires proper connection management**
- **Increased complexity due to multiple layers**
- **Performance depends on query optimization**

◈ 12. Conclusion

In this phase, a structured relational database and DAO layer were successfully implemented for the Clinic Appointment System. The use of JDBC, prepared statements, and transaction management ensures secure, consistent, and efficient database operations. The separation of concerns achieved through the DAO pattern enhances scalability and maintainability of the system.

◈ 13. References

- **Java JDBC Documentation**
- **Oracle Database Concepts**
- **OOP Design Principles (DAO Pattern)**